

Parâmetros

**args e **kwargs*

E quando a função precisa aceitar **quantos argumentos vierem**? `*args` recolhe os posicionais numa tupla; `**kwargs` recolhe os nomeados num dicionário. E os mesmos `*` e `**`, na chamada, fazem o contrário: espalham uma lista ou um dicionário em argumentos. Nesta aula: os dois lados, a ordem correta dos parâmetros e o repasse de argumentos.

`*args` → tupla

`**kwargs` → dict

Ordem dos parâmetros

`*lista` na chamada

`**dict` na chamada

Repassar argumentos

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Arbitrary Argument Lists, Unpacking Argument Lists, Keyword Arguments), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Quantos posicionais *vierem*

args.py

```
def somar(*numeros):          # junta os posicionais numa tupla
    print(type(numeros), numeros)
    return sum(numeros)

print(somar(1, 2))
print(somar(1, 2, 3, 4, 5))
print(somar())
```

SAÍDA

```
<class 'tuple'> (1, 2)
3
<class 'tuple'> (1, 2, 3, 4, 5)
15
<class 'tuple'> ()
0
```

- ✓ O `*` antes do nome recolhe os argumentos posicionais numa **tupla** (aula 10)
- ✓ Zero argumentos também vale: a tupla fica vazia
- ✓ O nome `args` é só convenção: `*numeros` é mais claro quando dá

O `print()` funciona assim: aceita quantos valores você passar.

Quantos nomeados *vierem*

kwargs.py

```
def perfil(**dados):
    # junta os nomeados num dict
    print(type(dados))
    for chave, valor in dados.items():
        print(f" {chave}: {valor}")

perfil(nome="Ana", cidade="Recife", linguagem="Python")
```

SAÍDA

```
<class 'dict'>
nome: Ana
cidade: Recife
linguagem: Python
```

- ✓ ****** recolhe os argumentos **nomeados** num **dicionário** (aula 12)
- ✓ O nome do argumento vira a chave; o valor, o valor
- ✓ Útil para opções livres: filtros, configurações, campos extras

O exemplo do guia (cap. 7): `config(tema='escuro', idioma='pt-BR')` .

A ordem *dos parâmetros*

combinados.py

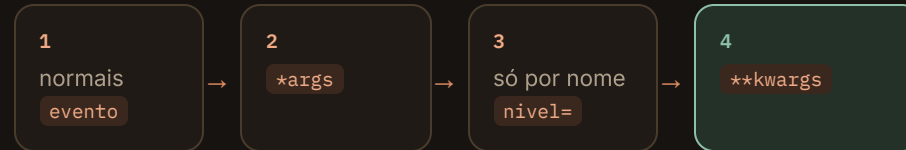
```
def registrar(evento, *detalhes, nivel="INFO", **extras):  
    print(f"[{nivel}] {evento}", detalhes, extras)  
  
registrar("login")  
registrar("erro", "timeout", 30, nivel="ERRO", usuario="ana")
```

```
[INFO] login () {}  
[ERRO] erro ('timeout', 30) {'usuario': 'ana'}
```

ordem_errada.py

```
def f(**kwargs, *args):  
    pass
```

```
File "ordem_errada.py", line 1  
    def f(**kwargs, *args):  
              ^  
SyntaxError: arguments cannot follow var-keyword argument
```



✓ Depois de `*args`, os parâmetros só aceitam nome (como o `*` da aula 18)

✓ `**kwargs` é sempre o último

Espalhando *listas e dicionários*

desempacotar.py

```
def apresentar(nome, idade, cidade):  
    print(f"{nome}, {idade}, {cidade}")  
  
dados = ["Ana", 29, "Recife"]  
apresentar(*dados)           # * espalha a lista  
info = {"cidade": "Natal", "nome": "Bia", "idade": 31}  
apresentar(**info)           # ** espalha o dict  
print(*["a", "b", "c"], sep="-")  # funciona no print também
```

SAÍDA

```
Ana, 29, Recife  
Bia, 31, Natal  
a-b-c
```

ONDE	*	**
Na definição <code>def f(...)</code>	Recolhe em tupla	Recolhe em dicionário
Na chamada <code>f(...)</code>	Espalha lista/tupla em posicionais	Espalha dicionário em nomeados

Com `**info`, a ordem das chaves não importa: cada valor vai para o parâmetro de mesmo nome.

Repassando *sem conhecer*

repassar.py

```
def cumprimentar(nome, cumprimento="Olá"):
    return f"{cumprimento}, {nome}!"

def gritar(*args, **kwargs):    # repassa tudo, sem conhecer
    return cumprimentar(*args, **kwargs).upper()

print(gritar("Ana"))
print(gritar("Bia", cumprimento="Bom dia"))
```

SAÍDA

```
OLÁ, ANA!
BOM DIA, BIA!
```

- ✓ `gritar` recebe qualquer coisa e entrega tudo para `cumprimentar`
- ✓ Se `cumprimentar` ganhar um parâmetro novo, `gritar` não precisa mudar
- ✓ É o padrão dos **decoradores** (capítulo 15 do guia):
`wrapper(*args, **kwargs)`

Não abuse: uma função com `**kwargs` em tudo esconde quais opções existem. Prefira parâmetros com nome quando você sabe quais são.

Média de *quantas notas vierem*

Enunciado: crie `media(*notas, casas=1)` que aceite qualquer quantidade de notas, arredonde com `casas` e devolva `None` se nenhuma nota vier.

Uma resposta possível

```
# Exercício: média de quantas notas vierem
def media(*notas, casas=1):
    if not notas:
        return None
    return round(sum(notas) / len(notas), casas)

print(media(7, 8, 9))
print(media(6.5, 7.25, casas=2))
print(media())
```

SAÍDA

```
8.0
6.88
None
```

- ✓ `casas` vem depois de `*notas` : só por nome
- ✓ `if not notas` : tupla vazia é falsa (aula 13)
- ✓ Desafio: chame com uma lista pronta usando `*`

Teste o que *ficou*

1. Dentro da função, `*args` é um(a):

A) Lista

B) Tupla

C) Dicionário

D) Set

B.

2. E `**kwargs`?

A) Lista

B) Tupla

C) Dicionário

D) String

C.

3. Qual definição é válida?

A) `def f(**k, *a)`

B) `def f(*a, **k)`

C) `def f(**k, x)`

D) `def f(*a, *b)`

B. `**kwargs` por último.

4. Com `d = {"a": 1, "b": 2}`, `f(**d)` equivale a:

A) `f(d)`

B) `f(a=1, b=2)`

C) `f("a", "b")`

D) `f(1, 2)` sempre

B. Espalha em nomeados.

Quantos argumentos *vierem*

Na definição, `*args` recolhe posicionais numa tupla e `**kwargs` recolhe nomeados num dicionário, nessa ordem e com `**kwargs` por último. Na chamada, `*` e `**` espalham listas e dicionários. Juntos, permitem repassar argumentos sem conhecê-los.

`*args`

`**kwargs`

Ordem

`f(*lista)` · `f(**dict)`

Repasse

Próxima aula → 21 · Função Lambda

Fontes: docs.python.org/3/tutorial/controlflow.html (Arbitrary Argument Lists, Unpacking Argument Lists, Keyword Arguments), consultados em outubro de 2026.
Exemplos em [exemplos/](#), executados com Python 3.14.4.